

Capítulo 16

Deployment

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 16 — Deployment

Objetivo de aprendizagem: servir o modelo por HTTP e descobrir, medindo, as duas decisões que definem um serviço de LLM — **streaming** (o que o usuário sente) e **batching** (o que o servidor entrega). São coisas diferentes, e otimizar uma não otimiza a outra.

Pré-requisitos: Capítulos 12 (KV-cache, prefill vs decode) e 13 (quantização).

Arquivos:

- `servidor.py` — servidor HTTP com streaming, biblioteca padrão apenas
- `carga.py` — mede TTFT, latência e vazão sob concorrência
- `servidor_batch.py` — o mesmo servidor, agrupando requisições
- `exercicios.md` — exercícios

Sem framework, de propósito. Nada de FastAPI ou uvicorn — só `http.server`. O assunto do capítulo são as decisões de serviço, e um framework as esconde atrás de decoradores. Você vai escrever o **Transfer-Encoding: chunked** na mão.

1. Um modelo treinado não é um serviço

Até aqui o modelo viveu dentro de scripts. Servir é outra coisa, e as perguntas mudam:

No treino você pergunta	Servindo você pergunta
a loss caiu?	quanto tempo até o usuário ver algo?
quanto tempo leva uma época?	quantos usuários simultâneos cabem?
cabe na memória?	o que acontece quando dez chegam juntos?

Nenhuma dessas aparece numa curva de treino, e é por isso que este capítulo mede coisas que nenhum capítulo anterior mediu.

2. Streaming: a otimização que não acelera nada

O servidor tem dois endpoints que produzem **o mesmo texto**:

```
POST /gerar devolve a resposta inteira, de uma vez
POST /gerar/stream devolve token por token, conforme são produzidos
```

O que muda é quando o primeiro byte chega:

Endpoint	TTFT	Total
/gerar	0,115 s	0,115 s
/gerar/stream	0,079 s	0,125 s

O **total é o mesmo** — tem de ser, o servidor faz o mesmo trabalho. O que muda é o **TTFT** (*time to first token*), e é ele que o usuário sente como "travou" ou "respondeu".

Por que isso é possível

Porque a geração é **autorregressiva**: os tokens existem um a um, em ordem. O **gerar** do servidor é um **gerador Python** — ele não devolve o texto pronto, cede cada token no instante em que existe:

```
for t in gerar_tokens(ids, n):
    pedaco = decodificar([t]).encode("utf-8")
    self.wfile.write(f"{len(pedaco):X}\r\n".encode() + pedaco + b"\r\n")
    self.wfile.flush() # sem isto, o SO junta tudo e não há streaming
```

Aquele **flush()** é o capítulo inteiro em uma linha. Sem ele o sistema operacional acumula os bytes num buffer e entrega tudo junto — o código parece streaming e não é.

Um modelo que produzisse a resposta inteira de uma vez não teria o que transmitir aos poucos. O streaming é grátis porque a arquitetura já é sequencial.

(!) E a magnitude aqui é pequena, por um motivo

A diferença medida é de 0,04 s — uns 30%. É pouco, e a causa é que **este modelo é rápido demais para o streaming brilhar**: 40 tokens saem em 0,12 s, então o TTFT já era baixo e boa parte dele é custo de HTTP, não de geração.

O ganho do streaming é proporcional ao **tempo total de geração**. Num modelo de verdade, 500 tokens a 30 tok/s levam ~17 segundos — e o TTFT cai de **17 s para ~0,3 s**. É a diferença entre um sistema que parece travado e um que parece instantâneo.

Este capítulo mede o **mecanismo**. A magnitude vem da escala.

3. O que acontece quando vários clientes chegam juntos

O servidor usa **ThreadingHTTPServer**, então cada requisição roda numa thread. Parece que vai escalar. Medindo:

Clientes	Latência mediana	p95	Vazão total
1	0,15 s	0,15 s	268 tok/s
2	0,20 s	0,27 s	301 tok/s

4	0,32 s	0,50 s	320 tok/s
8	0,54 s	0,96 s	334 tok/s

Oito vezes mais clientes, 25% mais trabalho entregue. E a latência cresce 3,6x, com o p95 quase dobrando a mediana — há fila, e quem chega por último paga.

A causa está no `servidor.py`: existe uma **trava** em volta da geração, então as requisições se **revezam** no modelo. Os 25% de ganho vêm das partes que rodam fora da trava — HTTP, tokenização, JSON.

E vale ser preciso: **a trava não está errada**. Sem ela, várias threads chamariam o modelo ao mesmo tempo e disputariam os mesmos núcleos, o que costuma sair pior. O problema é outro — requisições concorrentes estão sendo tratadas como **independentes** quando poderiam viajar **juntas**.

4. Batching: as requisições viajam juntas

A solução vem direto do [Capítulo 12](#): o decode é limitado por **memória**. Ler os pesos do modelo custa o mesmo para 1 ou para 16 sequências — então processar 16 juntas custa quase o mesmo que processar 1.

Lá isso foi medido: gerar 16 sequências em paralelo custou **9,1x menos** que 16 gerações separadas. Aqui é a mesma economia, aplicada a requisições de usuários diferentes.

O desenho inverte quem manda:

```
cliente A --\ /--> tokens de A
cliente B ---+--> [fila] --> laço -----+----> tokens de B
cliente C --/ (1 thread) \--> tokens de C
```

Um **único thread** é dono do modelo. As requisições HTTP não geram nada — elas põem um pedido numa fila e esperam os tokens voltarem. O thread-dono junta quem estiver esperando, gera **um token para todos de uma vez**, distribui, e repete.

E quem termina **sai do lote na hora**, sem esperar os outros:

```
for p in [p for p in ativos if p.restantes <= 0]:
    p.saida.put(None)
ativos = [p for p in ativos if p.restantes > 0]
```

É isso que torna o batching **contínuo** em vez de estático — e é o que vLLM e TGI fazem, com muito mais cuidado.

5. O resultado

Vazão total (tokens/s):

Cientes	Simples (trava)	Com batching	Ganho
1	268	306	1,14x
2	301	434	1,44x
4	320	585	1,83x
8	334	755	2,26x

Latência (mediana / p95):

Cientes	Simples (trava)	Com batching
1	0,15 s / 0,15 s	0,13 s / 0,13 s
2	0,20 s / 0,27 s	0,18 s / 0,18 s
4	0,32 s / 0,50 s	0,27 s / 0,27 s
8	0,54 s / 0,96 s	0,42 s / 0,42 s

2,26x mais vazão com 8 clientes — e latência menor. Não houve troca: o batching ganhou nos dois eixos.

O detalhe mais bonito da tabela

Olhe o **p95** das duas colunas.

Com a trava, ele é quase o dobro da mediana (0,96 contra 0,54). Isso é uma **fila**: a maioria é atendida rápido e alguns esperam muito, porque chegaram por último.

Com batching, **p95 = mediana**. Todos avançam **no mesmo passo** — cada iteração produz um token para todo mundo. Não existe "último da fila", então não existe cauda.

Um p95 igual à mediana é a assinatura de um sistema onde ninguém espera pelos outros. Vale procurar por isso em qualquer serviço que você meça.

6. Um bug meu, e por que ele vale estar aqui

A primeira versão do `servidor_batch.py` era **mais lenta** que o servidor simples: 0,62 s de latência contra 0,15 s, e 293 tok/s contra 334.

A causa era uma linha:

```
ativos.append(FILA.get(timeout=None if not ativos else ESPERA_MS / 1000))
```

Ela esperava 8 ms por novas requisições **a cada passo de decode**. Com 40 tokens, isso somava **320 ms de espera pura** por requisição. O ganho do agrupamento existia e estava sendo inteiramente consumido pela espera.

A correção é uma regra que vale para qualquer laço de serviço:

Nunca bloqueie quando há trabalho a fazer. Bloqueie só quando estiver ocioso; quando houver fila, drene sem esperar.

```
if not ativos:
    ativos.append(FILA.get()) # ocioso: pode bloquear
while len(ativos) < LOTE_MAX:
    try: ativos.append(FILA.get_nowait()) # ocupado: nunca espera
    except queue.Empty: break
```

Vale registrar o que quase aconteceu: eu tinha uma tabela mostrando batching **pior** que a versão simples. Ela era verdadeira e teria ensinado a coisa errada, porque a causa não era o batching — era a minha espera de 8 ms.

7. O que este capítulo não faz

Um servidor de produção tem várias coisas que aqui foram simplificadas de propósito, e convém saber quais:

Simplificação	O que a produção faz
todas as sequências recortadas no mesmo comprimento	<i>padding</i> com máscara, ou paged attention
KV-cache recalculado a cada passo do lote	cache persistente por sequência
sem limite de fila, sem timeout de admissão	<i>backpressure</i> — recusar quando cheio é melhor que atender mal
sem autenticação, sem limite de taxa	ambos, sempre
um processo, uma máquina	réplicas atrás de um balanceador

A que mais dói é a primeira: recortar no menor comprimento é correto aqui porque todas as requisições do teste começam parecidas, e seria errado com prompts de tamanhos diferentes.

Este capítulo mostra o mecanismo do batching, não uma implementação de produção. A diferença entre os dois é o assunto de bibliotecas inteiras.

8. Resumo do capítulo

- **Streaming** não acelera nada e muda tudo: mesmo total, TTFT muito menor. O ganho é proporcional ao tempo de geração — pequeno aqui, enorme num modelo real.
- O `flush()` depois de cada token é o que separa streaming de streaming aparente.
- **Threads não escalam** um serviço de LLM: com a trava, 8 clientes rendem 25% mais vazão e 3,6x mais latência.
- **Batching escala**, porque o decode é limitado por memória — 2,26x de vazão e latência menor.
- **p95 = mediana** é a assinatura de um sistema sem fila. Meça o p95, não só a média.
- Num laço de serviço, **nunca bloqueie quando há trabalho** — foi o que fez o meu batching ficar mais lento que o servidor simples.

Próximo capítulo

Capítulo 17 — Multimodal. O último. Até aqui o modelo só viu tokens de texto. E se a mesma arquitetura pudesse receber uma imagem?

Exercícios — Capítulo 16 (Deployment)

Faça na ordem. Soluções comentadas em [solucoes/](#) — **tente antes de olhar**.

Estes exercícios precisam de **dois terminais**: um com o servidor de pé, outro rodando o cliente. Suba o servidor com `python servidor.py` e meça com `python carga.py`.

E1 — TTFT e latência (aquecimento)

Sem rodar:

1. O `/gerar` e o `/gerar/stream` produzem o mesmo texto e levam o mesmo tempo total. Então o que exatamente o streaming melhora? Descreva em termos do que o usuário observa.
 2. Por que o streaming é possível num LLM e não seria, digamos, num classificador de imagens que devolve uma etiqueta?
 3. O servidor chama `self.wfile.flush()` depois de cada token. O que aconteceria sem essa linha, e por que o código continuaria "parecendo" streaming?
-

E2 — Meça o seu próprio TTFT (importante)

1. Rode o `carga.py` e anote TTFT e total nos dois endpoints. Qual a razão entre eles?
 2. Agora peça **200 tokens** em vez de 40 (`TOKENS = 200` no `carga.py`). O que acontece com a razão TTFT/total nos dois casos? Explique a tendência.
 3. Extrapole: para um modelo que gera 30 tokens/s e uma resposta de 500 tokens, quanto seria o TTFT com e sem streaming? Isso muda a sua opinião sobre a magnitude medida no capítulo?
-

E3 — Por que threads não bastam (importante)

1. Rode `carga.py` contra o `servidor.py` e monte a tabela de vazão por número de clientes. A vazão cresce proporcionalmente? Quanto?
 2. Remova a trava (`TRAVA`) do `servidor.py` e meça de novo. A vazão melhorou? (Cuidado: o resultado pode surpreender — meça antes de opinar.)
 3. Explique, usando o que o Capítulo 12 estabeleceu sobre o decode, por que threads independentes não resolvem — e por que agrupar resolve.
-

E4 — O tamanho do lote

No `servidor_batch.py`, o `LOTE_MAX` é 16.

1. Meça a vazão com `--lote 1`, `--lote 4` e `--lote 16`, sempre com 8 clientes. Como ela varia?

2. `--lote 1` deveria reproduzir o comportamento do servidor sem batching. Reproduz? Se não, o que mais mudou entre os dois servidores?
3. A partir de que tamanho de lote o ganho satura? O que limita — e a resposta é a mesma do E4 do Capítulo 8?

E5 — A espera que custou caro

O capítulo relata um bug: esperar 8 ms por novos pedidos **a cada passo** de decode.

1. Reintroduza a espera e meça. Quanto ela custa por requisição de 40 tokens? A conta bate com `40 × 8 ms`?
2. Existe algum cenário em que essa espera **ajudaria**? (Dica: pense em requisições que chegam espaçadas em vez de todas juntas.)
3. Formule a regra geral que a correção segue, e dê um exemplo de outro sistema onde ela se aplica.

E6 — O p95 conta o que a média esconde

1. Compare mediana e p95 nos dois servidores, com 8 clientes. Por que eles são diferentes num caso e iguais no outro?
2. Um serviço tem latência média de 200 ms e p99 de 4 s. Descreva o que o usuário experimenta — e por que reportar só a média seria enganoso.
3. Que outra métrica de cauda você acompanharia num serviço de LLM, além do p95 de latência? (Pense no que mais pode ter cauda longa.)

E7 — Backpressure (desafio)

O servidor deste capítulo aceita requisições sem limite: a fila cresce indefinidamente.

1. Meça o que acontece com 64 clientes simultâneos. A latência ainda é utilizável?
2. Implemente um limite: se a fila passar de N pedidos, responda **HTTP 429** em vez de enfileirar. Escolha o N a partir da sua medição do item 1.
3. Argumente: por que recusar uma requisição pode ser **melhor** que atendê-la devagar? Pense do ponto de vista do cliente que já está sendo atendido.